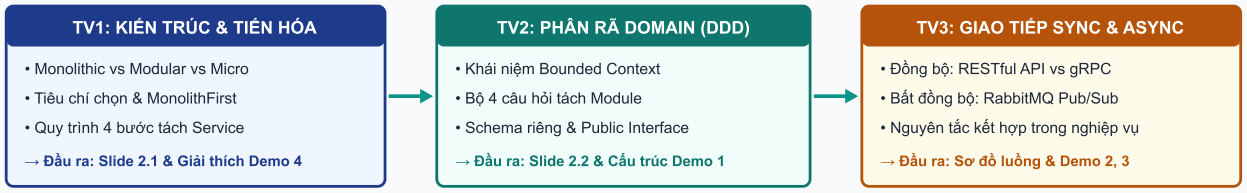
BUỔI 12 • KẾ HOẠCH NHÓM 3 NGƯỜI

Phân Công Nghiên Cứu Lý Thuyết Modular Monolithic

Tư duy thiết kế và con đường tiến hóa lên Microservice • Ngày: 2026-08-10

Để chuẩn bị hiệu quả cho buổi thuyết trình và thực hành demo bài toán Thương mại điện tử (E-commerce), toàn bộ nội dung lý thuyết tại Mục 2 được chia thành **3 mảng chuyên sâu** cho 3 thành viên (TV1, TV2, TV3). Cách phân bổ này đảm bảo cân bằng khối lượng nghiên cứu và liên kết trực tiếp với 4 phần demo bắt buộc.



Hình 1: Sơ đồ phân chia 3 trụ cột lý thuyết và mối liên hệ trực tiếp với đầu ra thuyết trình – demo.

1. Thành viên 1 (TV1): Tổng quan Kiến trúc & Lộ trình Tiến hóa

Mục 2.1 & 2.3

- **So sánh 3 mô hình kiến trúc (Mục 2.1):** Phân tích định nghĩa, ưu và nhược điểm của *Monolithic truyền thống* (một codebase, một process, dễ thành spaghetti code khi lớn), *Microservices* (độc lập deploy/scale/database nhưng phức tạp vận hành, cần CI/CD riêng, xử lý lỗi phân tán) và *Modular Monolithic* (vận hành đơn giản như Monolithic nhưng tổ chức module chặt chẽ qua interface).
- **Bộ tiêu chí lựa chọn thực tế & Nguyên tắc "MonolithFirst":** Đánh giá kiến trúc theo 3 yếu tố: *Quy mô đội ngũ* (nhóm nhỏ sinh viên vs nhiều team độc lập), *Nguồn lực hạ tầng & DevOps*, và *Giai đoạn sản phẩm* (mới thử nghiệm hay đã trưởng thành). Làm rõ tư duy bắt đầu từ Modular Monolithic và chỉ tách dần từng phần có lý do rõ ràng (kiến trúc lai – *hybrid architecture*).
- **Quy trình 4 bước tiến hóa lên Microservices (Mục 2.3):** Nghiên cứu lộ trình tách một module (ví dụ Payment vì bảo mật/compliance hoặc Inventory để scale mùa sale): (1) Tách schema ra database vật lý riêng; (2) Thay lời gọi interface in-process bằng network (REST/gRPC/RabbitMQ); (3) Đóng gói Docker image riêng (kế thừa Buổi 6, 7); (4) Tạo pipeline CI/CD riêng (kế thừa Buổi 10, 11).
- **Đầu ra phụ trách:** Soạn nội dung slide Mục 2.1 và đảm nhận phần giải thích trước lớp tại **Demo phần 4** (chỉ rõ những điểm cần thay đổi trong code khi tách module Inventory thành microservice độc lập mà không phải viết lại logic nghiệp vụ).

2. Thành viên 2 (TV2): Tư duy Phân rã Nghiệp vụ (Domain Decomposition)

Mục 2.2 – Trọng tâm

- **Nền tảng Bounded Context (Domain-Driven Design):** Nghiên cứu cách xác định một "ngữ cảnh nghiệp vụ" trọn vẹn có khái niệm, ngôn ngữ nghiệp vụ riêng biệt và hạn chế tối đa sự phụ thuộc vào chi tiết cài đặt bên trong của module khác.
- **Cấu trúc nội bộ & Quy tắc cô lập dữ liệu:** Mỗi module là một thư mục/project riêng, bên trong áp dụng lại *Layered/Clean Architecture* (Domain, Application, Infrastructure từ Buổi 4). Điều kiện tiên quyết: mỗi module phải có **schema/database riêng** (tuyệt đối không query hoặc JOIN thẳng vào bảng của module khác) và chỉ giao tiếp qua public interface (ví dụ `IInventoryService`) thay vì gọi trực tiếp vào Repository/DbContext của nhau.
- **Đầu ra phụ trách:** Xây dựng phần slide quan trọng nhất của buổi học (Mục 2.2) và làm cơ sở lý thuyết để tổ chức cây thư mục cho **Demo phần 1** (tối thiểu 3 module: Order, Inventory, Notification).

> **Câu hỏi 1: Ngôn ngữ nghiệp vụ riêng?**
>
> Nghiệp vụ có khái niệm khác biệt rõ không? Ví dụ: "SKU", "tồn kho khả dụng" thuộc *Inventory*, khác hẳn với "giỏ hàng", "mã giảm giá" của *Cart/Promotion*.

> **Câu hỏi 2: Ranh giới trách nhiệm tổ chức?**
>
> Trong doanh nghiệp thực tế, bộ phận nào phụ trách? Team Kho vận khác Team Marketing, khác Team Kế toán/Thanh toán – phản ánh đúng ranh giới module.

> **Câu hỏi 3: Tính độc lập của dữ liệu?**
>
> Dữ liệu của nghiệp vụ có thể đứng độc lập, không cần thực hiện truy vấn JOIN trực tiếp xuyên suốt với các bảng dữ liệu của module khác không?

> **Câu hỏi 4: Tốc độ thay đổi yêu cầu?**
>
> Tần suất thay đổi có khác nhau không? Ví dụ: *Promotion* biến đổi liên tục theo chiến dịch sale, trong khi *Catalog* tương đối ổn định nên cần tách riêng.

3. Thành viên 3 (TV3): Giao tiếp Đồng bộ & Bất đồng bộ

Mục 2.4, 2.5 & 2.6

- **Giao tiếp đồng bộ – RESTful API vs gRPC (Mục 2.4):** Làm rõ cơ chế gọi trực tiếp chờ phản hồi ngay. So sánh *REST* (HTTP/JSON dễ đọc, dễ debug, phù hợp public API cho frontend/đối tác) với *gRPC* (HTTP/2 + Protocol Buffers dạng binary gọn nhẹ, định nghĩa contract chặt chẽ qua file `.proto`, tối ưu cho giao tiếp nội bộ service-to-service). Phân tích rủi ro lỗi dây chuyền (*cascading failure*) khi gọi đồng bộ nhiều tầng.
- **Giao tiếp bất đồng bộ với RabbitMQ (Mục 2.5):** Nghiên cứu vai trò của Message Queue giúp phản hồi nhanh và giảm phụ thuộc chặt (*loose coupling*). Nắm vững các khái niệm cốt lõi: **Producer/Publisher**, **Consumer/Subscriber**, **Queue**, **Binding**, các loại **Exchange** (direct, topic, fanout) và cơ chế Publish/Subscribe qua *fanout exchange*.
- **Nguyên tắc lựa chọn & Kết hợp trong luồng E-commerce (Mục 2.6):** Khi cần kết quả ngay để quyết định bước tiếp theo (Order hỏi Inventory còn hàng không) → dùng **đồng bộ (REST/gRPC)**. Khi chỉ cần báo sự kiện vừa xảy ra (sau khi tạo đơn, phát event `OrderCreated` để Inventory trừ kho, Notification gửi email, Promotion cập nhật lượt dùng mã) → dùng **bất đồng bộ (RabbitMQ)**. Phụ trách lý thuyết cho **Demo phần 2 & phần 3**.

4. Phối hợp Chung & Bảng Phân công Tổng thể

Mục 3 & Mục 4

Sau khi từng thành viên hoàn thành nghiên cứu độc lập, **cả 3 thành viên** họp chung để áp dụng bộ 4 câu hỏi của TV2 nhằm phân tích 10 nghiệp vụ Thương mại điện tử (*Catalog, Cart, Order, Payment, Inventory, Shipping, Identity, Notification, Promotion, Review*), chốt sơ đồ giao tiếp đồng bộ/bất đồng bộ của TV3 và triển khai project demo theo đúng bảng phân bổ 36 Points dưới đây:

STT	HẠNG MỤC CÔNG VIỆC (LÝ THUYẾT & THỰC HÀNH DEMO)	NGƯỜI THỰC HIỆN	ƯỚC LƯỢNG
1	Nghiên cứu Monolithic / Modular Monolithic / Microservices và tiêu chí lựa chọn (Mục 2.1, 2.3)	**TV1**	3 Points
2	Nghiên cứu Bounded Context, tư duy phân rã domain – DDD ở mức cơ bản (Mục 2.2)	**TV2**	3 Points
3	Nghiên cứu RESTful vs gRPC cho giao tiếp đồng bộ & nguyên tắc chọn (Mục 2.4, 2.6)	**TV1 & TV3**	3 Points
4	Nghiên cứu RabbitMQ: Exchange, Queue, Binding, cơ chế Pub/Sub (Mục 2.5, 2.6)	**TV2 & TV3**	3 Points
5	Thiết kế bài toán E-commerce: liệt kê nghiệp vụ, xác định ranh giới module & luồng gọi	**Cả nhóm**	4 Points
6	Chuẩn bị nội dung slide thuyết trình (ưu tiên đầu tư kỹ nhất cho Mục 2.2)	**Nhóm trưởng**	2 Points
7	Dựng project Modular Monolithic demo (3 module, schema riêng, giao tiếp qua interface)	**TV3, TV1**	6 Points
8	Cài đặt RabbitMQ và viết Producer/Consumer demo (chứng minh tắt 1 Consumer không mất tin)	**TV2, TV3**	5 Points
9	Viết demo gọi đồng bộ REST/gRPC giữa Order và Inventory (đủ hàng & hết hàng)	**TV1**	4 Points
10	Tổng hợp, kiểm tra tích hợp và chạy thử toàn bộ kịch bản demo	**Cả nhóm**	2 Points
11	Luyện tập thuyết trình, phân bổ thời lượng và chuẩn bị phần giải thích tách Inventory	**Cả nhóm**	1 Point
Tổng khối lượng công việc toàn nhóm			**36 Points**

5. Lưu ý Quan trọng & Tài liệu Tham khảo

Mục 5 & Mục 6

Nguyên tắc cốt lõi cần nhấn mạnh khi báo cáo & Nguồn tài liệu chính thống

• **Lưu ý thuyết trình:** Không trình bày Microservices như "đích đến bắt buộc"; ranh giới module không có đáp án duy nhất mà phải giải thích bằng bộ câu hỏi mục 2.2; tuyệt đối không truy cập chéo DbContext/bảng dữ liệu giữa các module; làm rõ tính tin cậy của RabbitMQ khi Consumer tạm ngắt.

• **Tài liệu tra cứu:** RabbitMQ Docs (`rabbitmq.com/docs`), gRPC Docs (`grpc.io/docs`), Protocol Buffers Docs, bài viết *"MonolithFirst"* của Martin Fowler, kinh nghiệm Modular Monolith thực tế của Shopify và tài liệu nhập môn DDD / Bounded Context.